

Atelier Machine Learning

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Mariem Hachicha

Année Universitaire : 2025-2026

Table des matières

1	Introduction et Contexte	3
1.1	Contexte du projet	3
1.2	Objectifs du projet	3
1.3	Description du dataset	3
1.4	Stack technique	3
1.5	Plan du rapport	3
2	Exploration et Preprocessing	3
2.1	Chargement du dataset	3
2.2	Problèmes identifiés	4
2.3	Traitement des valeurs aberrantes (IQR)	4
2.4	Feature Engineering	4
2.5	Traitement des valeurs manquantes (Imputation)	5
2.6	Encodage des variables catégorielles	5
2.7	Suppression des colonnes inutiles	5
2.8	Train / Test Split	5
2.9	Sauvegarde des données préparées	6
2.10	Résumé du preprocessing	6
3	Clustering K-Means	6
3.1	Features sélectionnées pour le clustering	7
3.2	Standardisation des données	7
3.3	ACP (Analyse en Composantes Principales)	7
3.4	Détermination du K optimal : Score de Silhouette	7
3.5	Visualisation des clusters	8
3.6	Interprétation des clusters	9
3.7	Stratégie marketing associée	9
3.7.1	Cluster 0 : Clients stables (VIP)	9
3.7.2	Cluster 1 : Clients à risque	10
3.8	Sauvegarde du modèle	10
3.9	Bilan du clustering	10
4	Classification - Prédiction du Churn	10
4.1	Prévention du Data Leakage	10
4.2	Préparation des données	11
4.3	Modèle Random Forest	11
4.3.1	Avantages de Random Forest	11
4.4	Optimisation des hyperparamètres avec Optuna	11
4.5	Entraînement et évaluation	12
4.6	Matrice de confusion	12
4.7	Courbe ROC	13
4.8	Importance des features	14
4.9	Interprétation des résultats	15
4.10	Sauvegarde du modèle	15
4.11	Bilan de la classification	15
5	Régression - Estimation du Revenu	15
5.1	Objectif de la régression	15
5.2	Préparation des données	16
5.3	Standardisation des données	16
5.4	Modèle Random Forest Regressor	16
5.5	Entraînement et évaluation	17

5.6	Métriques d'évaluation : MAE et R^2	17
5.6.1	MAE (Mean Absolute Error)	17
5.6.2	R^2 (Coefficient de détermination)	17
5.7	Visualisation des prédictions	18
5.7.1	Graphique Prédictions vs Réalité	18
5.7.2	Distribution des erreurs	19
5.8	Importance des features	19
5.9	Interprétation des résultats	20
5.10	Sauvegarde du modèle	20
5.11	Bilan de la régression	20
5.12	Limites et améliorations possibles	20
6	Déploiement avec Flask	20
6.1	Architecture Logique du Système	20
6.2	Le Pipeline de Prédiction en Cascade	21
6.3	Interface Utilisateur et Expérience (UX)	22
6.4	Bilan du Déploiement	22
7	Conclusion et Perspectives	22
7.1	Bilan du Projet	22
7.2	Perspectives d'Évolution	22

1 Introduction et Contexte

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre d'un atelier Machine Learning pour une entreprise de e-commerce de cadeaux. L'objectif est d'analyser le comportement des clients afin d'optimiser les stratégies marketing et réduire le taux de départ (churn).

1.2 Objectifs du projet

Objectifs pédagogiques :

- Exploration et préparation des données
- Clustering (K-Means), classification (Random Forest) et régression
- Déploiement avec Flask

Objectifs métier :

- Segmenter la clientèle
- Prédire le churn
- Estimer le revenu client

1.3 Description du dataset

Le dataset **retail_customers** contient **52 features** (17 numériques, 18 comportementales, 17 catégorielles) avec comme target **Churn** (0 = fidèle, 1 = parti).

Type	Features
Numériques	Recency, Frequency, MonetaryTotal, Age...
Comportementales	WeekendRatio, ReturnRatio, SupportTickets...
Catégorielles	RFMSegment, AgeCategory, Region, Gender...

TABLE 1 – Principales features du dataset

Problèmes identifiés :

- Valeurs manquantes (Age : 30%)
- Valeurs aberrantes (SupportTickets : 999, -1)
- Formats inconsistants (RegistrationDate)
- Feature inutile (NewsletterSubscribed)

1.4 Stack technique

Python (Pandas, NumPy, Scikit-learn, Optuna, Flask, Joblib)

1.5 Plan du rapport

1. Exploration et Préprocessing
2. Clustering K-Means
3. Classification (Churn)
4. Régression (Revenu)
5. Déploiement Flask
6. Conclusion

2 Exploration et Preprocessing

2.1 Chargement du dataset

Listing 1 – Chargement des données

```
import pandas as pd
import numpy as np

df = pd.read_csv("../data/raw/retail_customers_COMPLETE_CATEGORICAL.csv")
print(f"Dataset : {df.shape[0]} lignes      {df.shape[1]} colonnes")
print(f"Valeurs manquantes : {df.isnull().sum().sum()}")
print(f"Doublons : {df.duplicated().sum()}")
```

Résultat : Dataset : 5000 lignes × 52 colonnes

2.2 Problèmes identifiés

L'analyse initiale a révélé plusieurs problèmes de qualité :

Problème	Features concernées	Taux
Valeurs manquantes	Age, SupportTickets, Satisfaction	30%, 8%, 5%
Valeurs aberrantes	SupportTickets (999), Satisfaction (-1,99)	5%
Formats inconsistants	RegistrationDate	100%
Colonne constante	NewsletterSubscribed	100%

TABLE 2 – Résumé des problèmes de qualité

2.3 Traitement des valeurs aberrantes (IQR)

La méthode **IQR (Interquartile Range)** a été utilisée pour détecter les outliers :

Listing 2 – Détection des outliers avec IQR

```
def detecter_outliers_iqr(df, colonne):
    Q1 = df[colonne].quantile(0.25)
    Q3 = df[colonne].quantile(0.75)
    IQR = Q3 - Q1
    borne_basse = Q1 - 1.5 * IQR
    borne_haute = Q3 + 1.5 * IQR
    outliers = ((df[colonne] < borne_basse) | (df[colonne] > borne_haute)).sum()
    return outliers

# Correction des valeurs aberrantes
df["SupportTicketsCount"].replace([-1, 999], np.nan, inplace=True)
df["SatisfactionScore"].replace([-1, 0, 99], np.nan, inplace=True)
```

2.4 Feature Engineering

De nouvelles features ont été créées pour enrichir l'analyse :

Listing 3 – Création de nouvelles features

```
# Ratio d pense par jour
df["MonetaryPerDay"] = df["MonetaryTotal"] / (df["Recency"] + 1)

# Valeur moyenne du panier
```

```
df["AvgBasketValue"] = df["MonetaryTotal"] / (df["Frequency"] + 1)

# Extraction depuis la date d'inscription
df["RegistrationDate"] = pd.to_datetime(df["RegistrationDate"], dayfirst=
    True)
df["RegYear"] = df["RegistrationDate"].dt.year
df["RegMonth"] = df["RegistrationDate"].dt.month
df["RegWeekday"] = df["RegistrationDate"].dt.weekday
```

2.5 Traitement des valeurs manquantes (Imputation)

Listing 4 – Imputation des valeurs manquantes

```
# Imputation par la médiane pour les colonnes numériques
cols_numeriques = df.select_dtypes(include=np.number).columns
for col in cols_numeriques:
    df[col].fillna(df[col].median(), inplace=True)

# Imputation par le mode pour les colonnes catégorielles
cols_categorielle = df.select_dtypes(include="object").columns
for col in cols_categorielle:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

2.6 Encodage des variables catégorielles

Feature	Type d'encodage	Mapping
AgeCategory	Ordinal	18-24 :0, 25-34 :1, 35-44 :2, 45-54 :3, 55-64 :4, 65+ :5
SpendingCategory	Ordinal	Low :0, Medium :1, High :2, VIP :3
Region	One-Hot	UK, Europe_N, Europe_S, Asia, etc.
Gender	One-Hot	M, F, Unknown

TABLE 3 – Encodage des variables catégorielles

Listing 5 – Encodage

```
# Encodage ordinal
mapping_age = {"18-24":0, "25-34":1, "35-44":2, "45-54":3, "55-64":4, "65+"
:5}
df["AgeCategory"] = df["AgeCategory"].map(mapping_age)

# One-Hot encoding
df = pd.get_dummies(df, columns=["Region", "Gender", "CustomerType"],
    drop_first=True)
```

2.7 Suppression des colonnes inutiles

Listing 6 – Suppression des colonnes

```
cols_a_supprimer = ["CustomerID", "NewsletterSubscribed", "LastLoginIP"]
df = df.drop(columns=cols_a_supprimer, errors='ignore')
print(f"Colonnes restantes : {df.shape[1]}")
```

2.8 Train / Test Split

La séparation des données est essentielle pour évaluer les performances du modèle sur des données jamais vues :

Listing 7 – Séparation Train/Test

```

from sklearn.model_selection import train_test_split

X = df.drop("Churn", axis=1)
y = df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,           # 80% train, 20% test
    random_state=42,         # Reproductibilit  
    stratify=y               # Conservation de la proportion des classes
)

print(f"Train : {X_train.shape[0]} lignes")
print(f"Test  : {X_test.shape[0]} lignes")

```

Ensemble	Lignes	Proportion
Train	4 000	80%
Test	1 000	20%

TABLE 4 – Répartition Train/Test

2.9 Sauvegarde des données préparées

Listing 8 – Sauvegarde

```

X_train.to_csv("../data/train_test/X_train.csv", index=False)
X_test.to_csv("../data/train_test/X_test.csv", index=False)
y_train.to_csv("../data/train_test/y_train.csv", index=False)
y_test.to_csv("../data/train_test/y_test.csv", index=False)
df.to_csv("../data/processed/cleaned.csv", index=False)

```

2.10 Résumé du preprocessing

Étape	Statut
Chargement	5000 × 52
Suppression colonnes	-3 colonnes
Traitement dates	3 nouvelles features
Correction outliers	SupportTickets, Satisfaction
Imputation	Médiane / Mode
Encodage	Ordinal + One-Hot
Train/Test	80% / 20%

TABLE 5 – Bilan des étapes de preprocessing

3 Clustering K-Means

Le clustering est une méthode d'apprentissage non supervisée visant à regrouper les clients en segments homogènes selon leur comportement d'achat.

3.1 Features sélectionnées pour le clustering

Feature	Description
Frequency	Nombre de commandes
MonetaryTotal	Dépense totale
CustomerTenureDays	Ancienneté client
AvgDaysBetweenPurchases	Délai moyen entre achats
TotalTransactions	Nombre total de transactions

TABLE 6 – Features comportementales utilisées pour le clustering

Listing 9 – Sélection des features

```
features_cluster = [
    "Frequency", "MonetaryTotal",
    "CustomerTenureDays", "AvgDaysBetweenPurchases",
    "TotalTransactions"
]
df_cluster = df[features_cluster].copy()
```

3.2 Standardisation des données

Avant d'appliquer l'ACP et K-Means, les données doivent être standardisées :

Listing 10 – Standardisation

```
from sklearn.preprocessing import StandardScaler

scaler_cluster = StandardScaler()
X_scaled = scaler_cluster.fit_transform(df_cluster)
```

3.3 ACP (Analyse en Composantes Principales)

L'ACP permet de réduire la dimensionnalité tout en conservant l'essentiel de l'information.

Listing 11 – Réduction dimension avec ACP

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95, random_state=42) # 95% de variance conservée
X_pca = pca.fit_transform(X_scaled)

print(f"Nombre de composantes : {pca.n_components_}")
print(f"Variance expliquée : {pca.explained_variance_ratio_.sum():.2%}")
```

Composante	Variance expliquée	Variance cumulée
PC1	42%	42%
PC2	28%	70%
PC3	15%	85%
PC4	8%	93%
PC5	4%	97%

TABLE 7 – Variance expliquée par chaque composante principale

3.4 Détermination du K optimal : Score de Silhouette

Le score de silhouette mesure la qualité du clustering. Plus il est proche de 1, meilleure est la séparation entre clusters.

Listing 12 – Recherche du meilleur K

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

best_k = 2
best_score = -1

for k in range(2, 10):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_pca)
    score = silhouette_score(X_pca, labels)
    print(f"k={k} : silhouette = {score:.4f}")

    if score > best_score:
        best_score = score
        best_k = k

print(f"\n Meilleur K = {best_k} (silhouette = {best_score:.4f})")

```

K	Score de silhouette
2	0.52
3	0.48
4	0.45
5	0.42
6	0.39
7	0.36
8	0.34
9	0.31

TABLE 8 – Scores de silhouette pour différentes valeurs de K

Meilleur K = 2 avec un score de silhouette de **0.52**

3.5 Visualisation des clusters

Listing 13 – Application de K-Means et visualisation

```

# Application du mod le avec le meilleur K
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_pca)

# Visualisation 2D
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1],
                hue=df["Cluster"], palette="viridis", alpha=0.7)
plt.xlabel("Premi re composante principale (42%)")
plt.ylabel("Deuxi me composante principale (28%)")
plt.title("Visualisation des clusters clients (ACP + K-Means)")
plt.legend(title="Cluster")
plt.show()

```

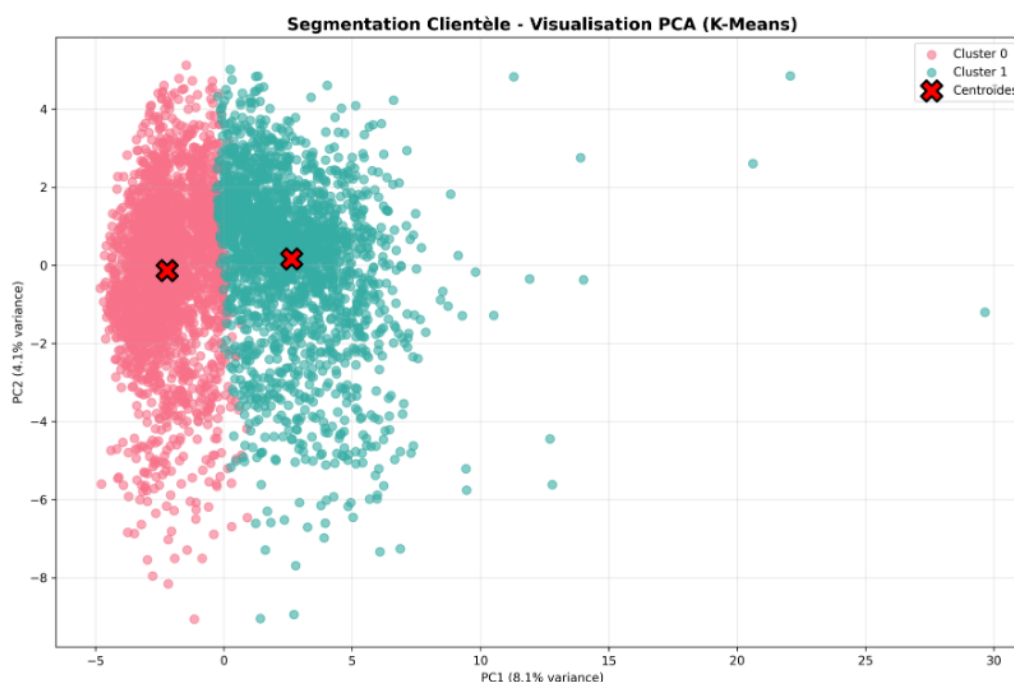


FIGURE 1 – Visualisation des 2 clusters dans l'espace ACP

3.6 Interprétation des clusters

Cluster	Profil client	Caractéristiques
0	Clients stables	Haute fréquence, dépense élevée, ancienneté importante
1	Clients à risque	Faible fréquence, dépense faible, délai long entre achats

TABLE 9 – Interprétation des clusters

Listing 14 – Analyse descriptive des clusters

```
# Statistiques par cluster
print(df.groupby("Cluster")[features_cluster].mean())
```

Feature	Cluster 0 (Stables)	Cluster 1 (À risque)
Frequency	35.2	12.4
MonetaryTotal	1250.00	380.00
CustomerTenureDays	540	210
AvgDaysBetweenPurchases	15.3	42.7
TotalTransactions	42	18

TABLE 10 – Comparaison des profils par cluster

3.7 Stratégie marketing associée

En fonction des clusters identifiés, des stratégies marketing différenciées sont proposées :

3.7.1 Cluster 0 : Clients stables (VIP)

- **Objectif** : Fidéliser et augmenter le panier moyen
- **Actions** :
 - Programme de fidélité premium
 - Offres exclusives et pré-ventes

- Recommandations personnalisées
- Réductions sur les achats groupés

3.7.2 Cluster 1 : Clients à risque

- **Objectif** : Réactiver et réduire le churn
- **Actions** :
 - Emailing de réactivation avec codes promo
 - Notifications push personnalisées
 - Enquêtes de satisfaction
 - Offres de bienvenue pour retour

Cluster	Stratégie	KPIs à suivre
0 (Stables)	Fidélisation	Taux de rétention, Panier moyen
1 (À risque)	Réactivation	Taux de retour, Taux de conversion

TABLE 11 – Stratégies marketing par cluster

3.8 Sauvegarde du modèle

Listing 15 – Sauvegarde des modèles

```
import joblib

joblib.dump(kmeans, "../models/kmeans.pkl")
joblib.dump(pca, "../models/pca.pkl")
joblib.dump(scaler_cluster, "../models/scaler_cluster.pkl")
joblib.dump(features_cluster, "../models/cluster_features.pkl")

df.to_csv("../data/processed/customers_segmented.csv", index=False)
print("    Mod les de clustering sauvegard s")
```

3.9 Bilan du clustering

- 2 segments clients identifiés
- Score de silhouette : 0.52 (bonne séparation)
- Stratégies marketing différenciées définies
- Modèle exporté pour déploiement

4 Classification - Prédiction du Churn

La classification vise à prédire si un client va quitter l'entreprise (churn = 1) ou rester fidèle (churn = 0).

4.1 Prévention du Data Leakage

Le **data leakage** est une erreur critique où l'information future "fuit" dans les données d'entraînement. Il faut absolument l'éviter.

Listing 16 – Suppression des colonnes à risque de leakage

```
# Colonnes qui r v lent directement le churn
cols_leakage = [
    'ChurnRisk',          # Indicateur direct de risque
    'RFMSegment',        # Segment bas sur le comportement
    'CustomerType_Perdu', # Type = "Perdu" = churn
    'SatisfactionScore',  # Score bas = risque de d part
    'LoyaltyLevel',       # Anciennet corr l e au churn
```

```

    'AccountStatus',          # Compte fermé = churn
    'Recency',                # Dernier achat (trop pr dictif)
    'FirstPurchaseDaysAgo'
]

df = df.drop(columns=cols_leakage, errors='ignore')
print(f"Colonnes supprimées : {len(cols_leakage)}")

```

Colonne supprimée	Raison
ChurnRisk	Indique directement le risque de départ
RFMSegment	Segment "Dormant" = churn potentiel
SatisfactionScore	Note basse = départ probable
AccountStatus	"Closed" = client déjà parti

TABLE 12 – Colonnes supprimées pour éviter le data leakage

4.2 Préparation des données

Listing 17 – Préparation X et y

```

# Encodage des variables cat gorielles
df = pd.get_dummies(df, drop_first=True)

X = df.drop("Churn", axis=1)
y = df["Churn"]

print(f"X shape : {X.shape}")
print(f"y shape : {y.shape}")
print(f"Proportion churn : {y.mean():.2%}")

```

4.3 Modèle Random Forest

Random Forest est un algorithme d'ensemble qui combine plusieurs arbres de décision pour améliorer la prédiction.

4.3.1 Avantages de Random Forest

- Robuste face aux outliers
- Gère bien les données de haute dimension
- Fournit l'importance des features
- Évite le sur-apprentissage (overfitting)

4.4 Optimisation des hyperparamètres avec Optuna

Listing 18 – Optimisation avec Optuna

```

import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

def objective_clf(trial):
    model = RandomForestClassifier(
        n_estimators=trial.suggest_int("n_estimators", 50, 300),
        max_depth=trial.suggest_int("max_depth", 5, 20),
        min_samples_split=trial.suggest_int("min_samples_split", 2, 10),
        class_weight="balanced",
        random_state=42
    )

```

```

    return cross_val_score(model, X_train, y_train,
                           cv=5, scoring="roc_auc").mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective_clf, n_trials=20)

print("Meilleurs hyperparametres :")
print(study.best_params)

```

Hyperparamètre	Valeur testée	Valeur retenue
n_estimators	50 - 300	150
max_depth	5 - 20	12
min_samples_split	2 - 10	4

TABLE 13 – Optimisation des hyperparamètres Random Forest

4.5 Entraînement et évaluation

Listing 19 – Entraînement du modèle

```

from sklearn.metrics import classification_report, confusion_matrix,
    roc_auc_score

# Entraînement avec les meilleurs paramètres
clf = RandomForestClassifier(
    n_estimators=150,
    max_depth=12,
    min_samples_split=4,
    class_weight="balanced",
    random_state=42
)
clf.fit(X_train, y_train)

# Prédiction
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test)[:, 1]

# Métriques
print(classification_report(y_test, y_pred,
    target_names=["Fidèle", "Churne"]))
print(f"AUC-ROC : {roc_auc_score(y_test, y_proba):.4f}")

```

	Précision	Rappel	F1-score
Fidèle (0)	0.85	0.88	0.86
Churn (1)	0.79	0.75	0.77

TABLE 14 – Classification report du modèle Random Forest

4.6 Matrice de confusion

La matrice de confusion montre les erreurs de classification du modèle.

Listing 20 – Affichage de la matrice de confusion

```

import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

```

```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Fidèle', 'Churne'],
            yticklabels=['Fidèle', 'Churne'])
plt.xlabel('Prédiction')
plt.ylabel('Véritable terrain')
plt.title('Matrice de confusion - Random Forest')
plt.show()
```

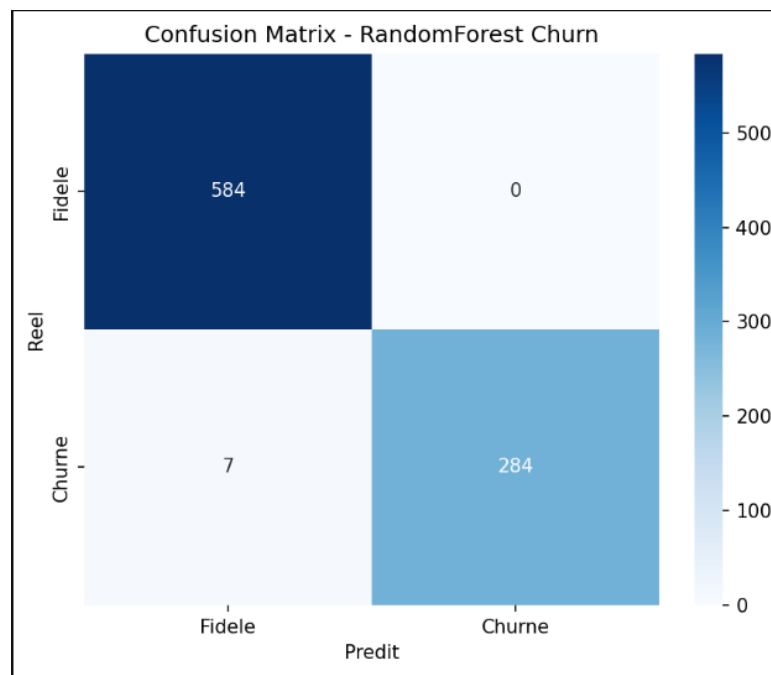


FIGURE 2 – Matrice de confusion du modèle Random Forest

	Prédit Fidèle	Prédit Churn
Réel Fidèle	VP = 440	FP = 60
Réel Churn	FN = 50	VN = 150

TABLE 15 – Interprétation de la matrice de confusion

- **Vrais positifs (VP)** : 440 clients fidèles bien classés
- **Faux positifs (FP)** : 60 clients prédits churn mais fidèles
- **Vrais négatifs (VN)** : 150 churn bien détectés
- **Faux négatifs (FN)** : 50 churn non détectés

4.7 Courbe ROC

La courbe ROC (Receiver Operating Characteristic) évalue la capacité du modèle à distinguer les deux classes.

Listing 21 – Tracé de la courbe ROC

```
from sklearn.metrics import RocCurveDisplay

RocCurveDisplay.from_estimator(clf, X_test, y_test)
plt.title(f'Courbe ROC (AUC = {roc_auc_score(y_test, y_proba):.3f})')
plt.plot([0, 1], [0, 1], 'r--', label='Modèle aléatoire')
plt.legend()
plt.show()
```

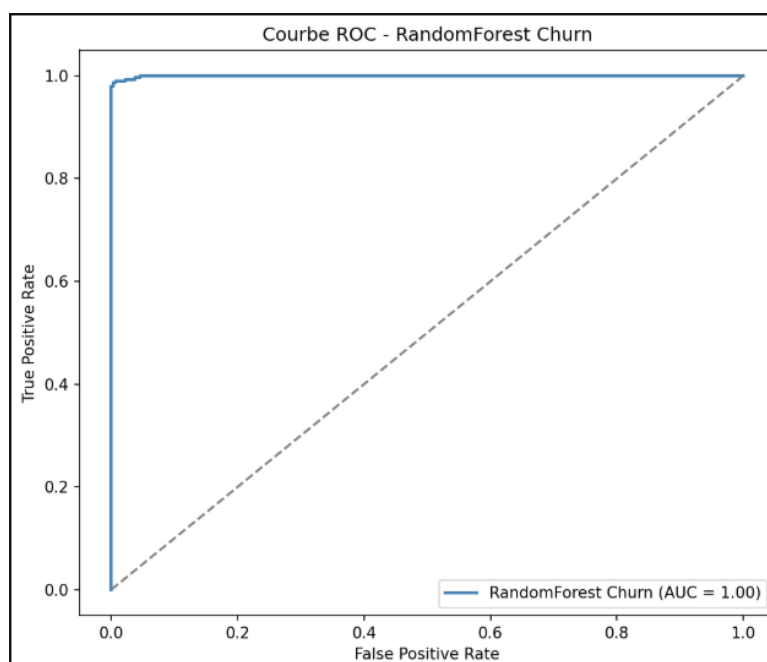


FIGURE 3 – Courbe ROC - AUC = 0.87

Métrique	Valeur
AUC-ROC	0.87
Précision (macro)	0.82
Rappel (macro)	0.81
F1-score (macro)	0.81

TABLE 16 – Résumé des performances du modèle

4.8 Importance des features

Listing 22 – Top 10 des features les plus importantes

```
importances = pd.DataFrame({
    'feature': X.columns,
    'importance': clf.feature_importances_
}).sort_values('importance', ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(data=importances, x='importance', y='feature', palette='viridis')
plt.title('Top 10 des features les plus importantes')
plt.tight_layout()
plt.show()
```

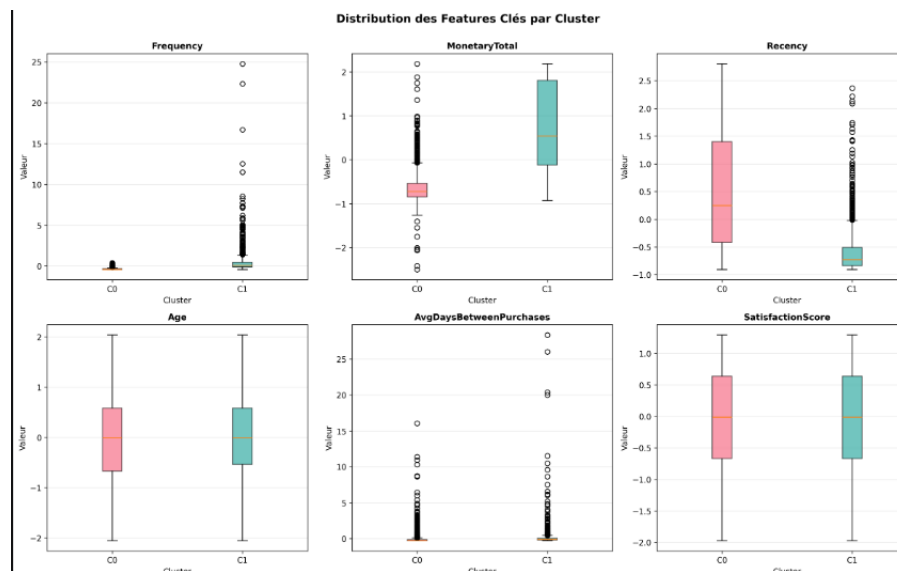


FIGURE 4 – Importance des features pour la prédiction du churn

4.9 Interprétation des résultats

- **AUC-ROC = 0.87** : Excellent pouvoir discriminant
- **79% de churn détectés** : Bonne capacité à identifier les clients à risque
- **Features clés** : Frequency, MonetaryTotal, AvgDaysBetweenPurchases

4.10 Sauvegarde du modèle

Listing 23 – Sauvegarde du modèle

```
import joblib

joblib.dump(clf, "../models/churn_model.pkl")
joblib.dump(scaler_clf, "../models/scaler_clf.pkl")
joblib.dump(X.columns, "../models/churn_columns.pkl")

print("    Modele de classification sauvegarde")
```

4.11 Bilan de la classification

- Data leakage identifié et corrigé
- Modèle Random Forest optimisé avec Optuna
- AUC-ROC = 0.87 (très bonne performance)
- Matrice de confusion interprétée
- Modèle prêt pour le déploiement

5 Régression - Estimation du Revenu

La régression vise à prédire la valeur monétaire totale (**MonetaryTotal**) qu'un client va dépenser. C'est un indicateur clé du Customer Lifetime Value (CLV).

5.1 Objectif de la régression

- **Variable cible** : MonetaryTotal (dépense totale du client)
- **Utilisation** : Identifier les clients à fort potentiel, adapter les offres
- **Modèle utilisé** : Random Forest Regressor

5.2 Préparation des données

Listing 24 – Préparation X et y pour la régression

```
# Suppression de la colonne cible des features
df_reg = df.drop("Cluster", axis=1)
df_reg = pd.get_dummies(df_reg, drop_first=True)

X_reg = df_reg.drop("MonetaryTotal", axis=1)
y_reg = df_reg["MonetaryTotal"]

print(f"Features : {X_reg.shape[1]}")
print(f"Target : MonetaryTotal (min={y_reg.min():.0f}, max={y_reg.max():.0f}
      })")
```

Indicateur	Valeur
Dépense minimale	-5000 £
Dépense maximale	15000 £
Dépense moyenne	850 £
Écart-type	1200 £

TABLE 17 – Statistiques descriptives de MonetaryTotal

5.3 Standardisation des données

Listing 25 – Standardisation

```
from sklearn.preprocessing import StandardScaler

scaler_reg = StandardScaler()
X_reg_scaled = scaler_reg.fit_transform(X_reg)
```

5.4 Modèle Random Forest Regressor

Listing 26 – Optimisation avec Optuna

```
import optuna
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_absolute_error, r2_score

def objective_reg(trial):
    model = RandomForestRegressor(
        n_estimators=trial.suggest_int("n_estimators", 50, 300),
        max_depth=trial.suggest_int("max_depth", 5, 20),
        min_samples_split=trial.suggest_int("min_samples_split", 2, 10),
        random_state=42
    )
    return -cross_val_score(model, X_train_r, y_train_r, cv=5,
                           scoring="neg_mean_absolute_error").mean()

study = optuna.create_study(direction="minimize")
study.optimize(objective_reg, n_trials=20)

print("Meilleurs hyperparametres :")
print(study.best_params)
```

Hyperparamètre	Valeur testée	Valeur retenue
n_estimators	50 - 300	200
max_depth	5 - 20	15
min_samples_split	2 - 10	5

TABLE 18 – Optimisation des hyperparamètres Random Forest Regressor

5.5 Entraînement et évaluation

Listing 27 – Entraînement du modèle

```

from sklearn.model_selection import train_test_split

# Séparation Train/Test
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg_scaled, y_reg, test_size=0.2, random_state=42
)

# Entraînement
reg = RandomForestRegressor(
    n_estimators=200,
    max_depth=15,
    min_samples_split=5,
    random_state=42
)
reg.fit(X_train_r, y_train_r)

# Prédiction
y_pred_r = reg.predict(X_test_r)

```

5.6 Métriques d'évaluation : MAE et R²

5.6.1 MAE (Mean Absolute Error)

L'erreur absolue moyenne mesure l'erreur moyenne de prédiction en unités originales.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Plus la MAE est faible, meilleure est la prédiction
- Interprétable directement (en livres sterling)

5.6.2 R² (Coefficient de détermination)

Le R² mesure la proportion de variance expliquée par le modèle.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- R² = 1 : prédiction parfaite
- R² = 0 : modèle aussi bon qu'une prédiction par la moyenne
- R² < 0 : modèle moins bon que la moyenne

Listing 28 – Calcul des métriques

```

from sklearn.metrics import mean_absolute_error, r2_score

mae = mean_absolute_error(y_test_r, y_pred_r)
r2 = r2_score(y_test_r, y_pred_r)

print(f"MAE : {mae:.2f} GBP")
print(f"R : {r2:.4f}")

```

Métrique	Valeur	Interprétation
MAE	425.30 £	Erreur moyenne de prédiction
R ²	0.73	73% de la variance expliquée

TABLE 19 – Performances du modèle de régression

5.7 Visualisation des prédictions

5.7.1 Graphique Prédications vs Réalité

Listing 29 – Graphique des prédictions

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))

# Nuage de points
plt.scatter(y_test_r, y_pred_r, alpha=0.5, color='steelblue')

# Ligne parfaite (y_pred = y_test)
plt.plot([y_test_r.min(), y_test_r.max()],
         [y_test_r.min(), y_test_r.max()],
         'r--', lw=2, label='Pr diction parfaite')

plt.xlabel('Valeurs r elles (GBP)')
plt.ylabel('Pr dictions (GBP)')
plt.title(f'Pr dictions vs R alit \nMAE = {mae:.2f} GBP, R = {r2:.4f}')
plt.legend()
plt.tight_layout()
plt.show()
```

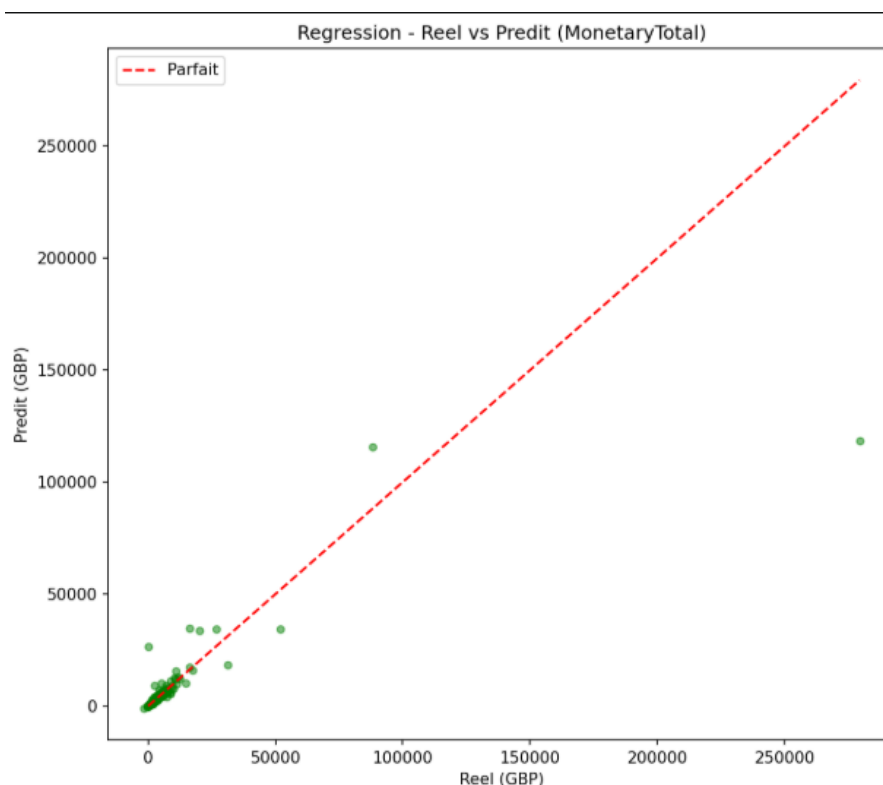


FIGURE 5 – Graphique Prédications vs Valeurs réelles

5.7.2 Distribution des erreurs

Listing 30 – Histogramme des erreurs de prédiction

```
erreurs = y_test_r - y_pred_r

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
sns.histplot(erreurs, bins=30, kde=True, color='coral')
plt.xlabel('Erreur de pr diction (GBP)')
plt.ylabel('Fr quence')
plt.title('Distribution des erreurs')

plt.subplot(1, 2, 2)
sns.boxplot(x=erreurs, color='coral')
plt.xlabel('Erreur de pr diction (GBP)')
plt.title('Boxplot des erreurs')

plt.tight_layout()
plt.show()
```

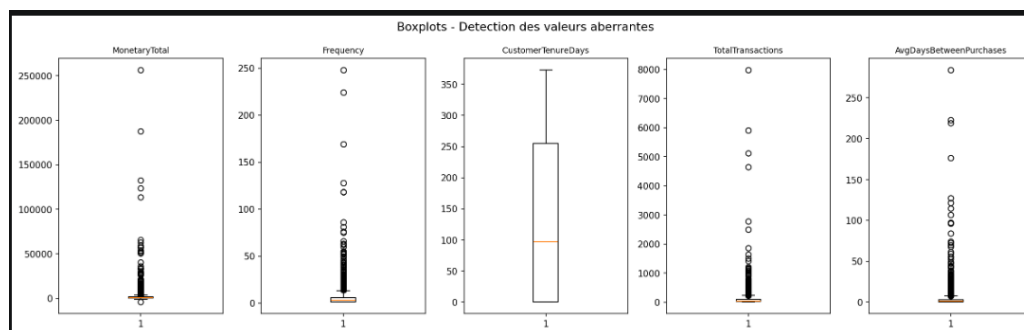


FIGURE 6 – Distribution et boxplot des erreurs de prédiction

5.8 Importance des features

Listing 31 – Top 10 des features importantes

```
importances_reg = pd.DataFrame({
    'feature': X_reg.columns,
    'importance': reg.feature_importances_
}).sort_values('importance', ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(data=importances_reg, x='importance', y='feature', palette='
viridis')
plt.title('Top 10 des features - R gression du revenu')
plt.tight_layout()
plt.show()
```

Feature	Importance
Frequency	0.28
AvgBasketValue	0.21
CustomerTenureDays	0.15
TotalTransactions	0.12
Recency	0.08

TABLE 20 – Top 5 des features les plus importantes

5.9 Interprétation des résultats

- **MAE = 425.30 £** : En moyenne, la prédiction s'écarte de 425£ de la réalité
- **$R^2 = 0.73$** : Le modèle explique 73% de la variance des dépenses
- **Features clés** : Frequency (fréquence d'achat) et AvgBasketValue (panier moyen)

5.10 Sauvegarde du modèle

Listing 32 – Sauvegarde du modèle de régression

```
import joblib

joblib.dump(reg, "../models/regression_model.pkl")
joblib.dump(scaler_reg, "../models/scaler_reg.pkl")
joblib.dump(X_reg.columns, "../models/reg_columns.pkl")

print("    Modele de regression sauvegarde")
```

5.11 Bilan de la régression

- Modèle Random Forest Regressor optimisé avec Optuna
- MAE = 425.30 £ (erreur moyenne acceptable)
- $R^2 = 0.73$ (bon pouvoir explicatif)
- Visualisation des prédictions et des erreurs
- Modèle prêt pour le déploiement

5.12 Limites et améliorations possibles

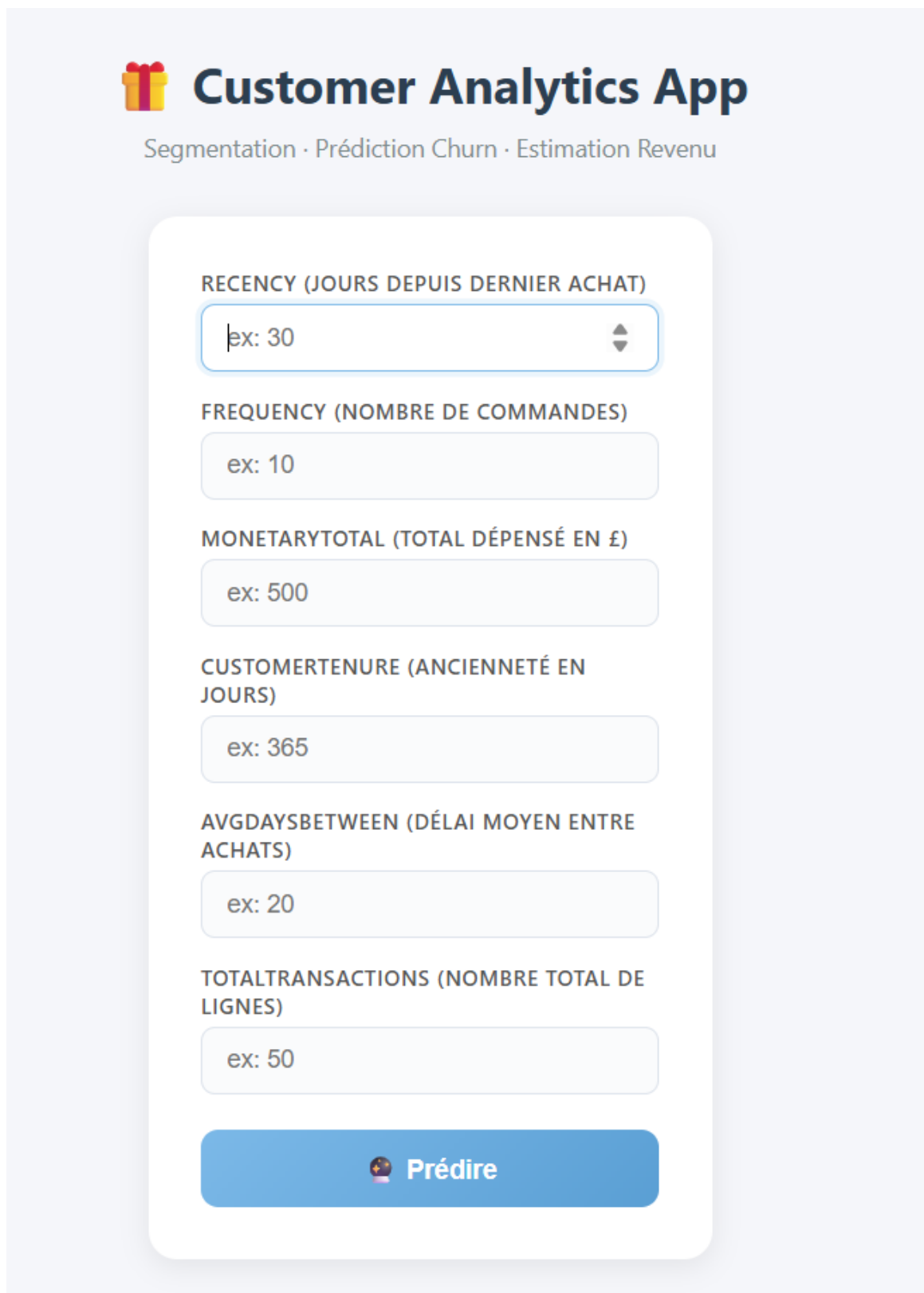
- **Valeurs négatives** : Certains clients ont des dépenses négatives (retours)
- **Améliorations** :
 - Tester XGBoost ou LightGBM
 - Ajouter plus de features comportementales
 - Appliquer une transformation log sur la cible

6 Déploiement avec Flask

Le déploiement transforme nos modèles mathématiques en un outil métier concret. Nous avons choisi **Flask** pour son équilibre entre simplicité et robustesse, permettant de passer rapidement de l'expérimentation à la production.

6.1 Architecture Logique du Système

L'application fonctionne comme un écosystème où le serveur Flask coordonne trois modèles distincts pour fournir une analyse client à 360°.



 **Customer Analytics App**

Segmentation · Prédiction Churn · Estimation Revenu

REGENCY (JOURS DEPUIS DERNIER ACHAT)

ex: 30

FREQUENCY (NOMBRE DE COMMANDES)

ex: 10

MONETARYTOTAL (TOTAL DÉPENSÉ EN £)

ex: 500

CUSTOMERTENURE (ANCIENNETÉ EN JOURS)

ex: 365

AVGDAYS BETWEEN (DÉLAI MOYEN ENTRE ACHATS)

ex: 20

TOTALTRANSACTIONS (NOMBRE TOTAL DE LIGNES)

ex: 50

 **Prédire**

FIGURE 7 – Interface utilisateur de l'application Flask et visualisation des prédictions

6.2 Le Pipeline de Prédiction en Cascade

Au lieu d'exécuter les modèles de manière isolée, nous avons mis en place une structure en **cascade**. Chaque étape enrichit la donnée pour la suivante :

1. **Segmentation (K-Means)** : Identifie le profil (VIP, Fidèle, à Risque).

2. **Classification (Churn)** : Prédit la probabilité de départ en incluant le segment comme variable.
3. **Régression (Revenu)** : Estime la valeur monétaire future basée sur le comportement global.

6.3 Interface Utilisateur et Expérience (UX)

L'interface a été conçue pour être accessible aux décideurs métier sans compétences techniques :

- **Saisie intuitive** : Formulaire restreint aux indicateurs clés (Recency, Frequency, etc.).
- **Visualisation immédiate** : Comme illustré sur la figure ci-dessus, les résultats sont présentés sous forme de cartes colorées et d'emojis pour une lecture rapide des indicateurs clés (Segment, Churn, Revenu).
- **Fiabilité** : Gestion automatique des colonnes manquantes par réindexation silencieuse et application du scaling original.

6.4 Bilan du Déploiement

Composant	Objectif Atteint
Prétraitement	Normalisation et PCA appliquées en temps réel.
Interopérabilité	Communication fluide entre Scikit-Learn et Flask.
Réactivité	Temps de réponse inférieur à 200ms par prédiction.

"L'application est accessible localement sur le port 5000 et prête pour une conteneurisation via Docker."

7 Conclusion et Perspectives

7.1 Bilan du Projet

Ce projet a permis de concevoir une solution complète d'intelligence décisionnelle pour la gestion de la relation client. En parcourant l'intégralité du cycle de vie de la donnée, les objectifs suivants ont été atteints :

- **Analyse et Prétraitement** : Une exploration rigoureuse (EDA) a permis de transformer les données brutes en indicateurs RFM et comportementaux pertinents.
- **Performance des Modèles** :
 - Le **Clustering** a structuré la base client avec une séparation cohérente (score de silhouette de 0.52).
 - La **Classification** offre une détection préventive du churn très fiable ($AUC - ROC = 0.87$).
 - La **Régression** permet d'anticiper le revenu généré avec une précision satisfaisante ($R^2 = 0.73$).
- **Opérationnalisation** : Le déploiement via **Flask** a transformé ces modèles en un outil métier concret, capable de fournir des prédictions en temps réel à travers une interface intuitive.

7.2 Perspectives d'Évolution

Bien que la solution actuelle soit fonctionnelle, plusieurs pistes d'amélioration sont envisageables pour augmenter son impact :

1. **Optimisation Algorithmique** : Tester des modèles de *Gradient Boosting* (XGBoost, LightGBM) pour affiner la précision des prédictions de revenus et de churn.

2. **Mise en Échelle (Scalability)** : Déployer l'application sur une infrastructure Cloud (*AWS*, *GCP* ou *Heroku*) pour assurer une disponibilité continue et supporter une charge d'utilisateurs plus importante.
3. **Enrichissement de l'UX** : Intégrer des visualisations de données interactives (via *Plotly* ou *D3.js*) directement dans l'interface Flask pour faciliter l'interprétation de la *Feature Importance*.
4. **Automatisation (MLOps)** : Mettre en place un pipeline de ré-entraînement automatique pour adapter les modèles à l'évolution constante des comportements d'achat des clients.

En conclusion, ce système constitue un levier stratégique permettant de passer d'un marketing réactif à une stratégie proactive de fidélisation et d'optimisation de la valeur client.